IBM Hackathon Submission

Technology Statement and Development Process

Vyuit / Doori
Zohar Yaffe | Team ZOHA Y | November 2, 2026

Development approach and working with Bob

I developed the existing Vyuit app in Lovable before the hackathon. During the hackathon, I created a separate branch in the existing GitHub repository and connected Bob to the project, so I could develop the agent separately from the existing work on the app.

I created an agent in IBM watsonx Orchestrate and used Build with Bob. I worked with Bob to define Doori's role, conversational style, scope of responsibility, agent and knowledge files, and initial checks. I defined the professional knowledge, inspection sequence and product decisions based on my architectural experience.

Use of AI

watsonx Orchestrate runs the agent and provides access to its knowledge base. The model's intended role is to conduct a conversation, explain instructions and terminology, and help formulate factual findings based on information provided by the user.

The agent instructions define response boundaries, including avoiding invented standards or findings and leaving the decision to document a defect with the user.

What works and what remains incomplete

The original agent and knowledge base were imported into the cloud, and conversations, knowledge retrieval and the integration of the chat widget into the app were tested. Detailed instructions were developed for Bathroom 1. Local updates were also made to the agent and knowledge files; basic file checks do not confirm that all updates have been imported into the cloud and work there.

The submission includes separately named agent and knowledge-base configuration files and the associated knowledge document. These submission files have not been imported and tested as a separate cloud deployment.

The main limitation is consistently maintaining the inspection sequence. Following a recommendation from an IBM developer, we planned two tools: get_next_item to retrieve the next unanswered checklist item, and record_answer to save an answer and mark the item as completed. The tools have not been implemented, imported or tested. Shared persistent storage remains an unresolved design requirement.

No connection has been implemented that allows Doori to create or update defect records in the app's database. Users document findings through the app's existing interface.

Transferring suggested wording directly into the defect form, preserving drafts during that transfer and verifying separation between users' chat sessions remain follow-up tasks. I chose to submit the existing prototype and explicitly present its limitations, without changing the app's infrastructure under time pressure.

Interactive demonstration

A separate HTML demo illustrates the intended user experience, including guided conversation, editing a finding, simulated saving and a summary. Its conversation is scripted and its saving is simulated. It is not connected to watsonx Orchestrate or the app's database, and it does not retain findings after reloading. Photo attachment is available in the existing app but is not implemented in this demo.

Further character development

Initial visual design work on Doori has begun. Planned animation would convey waiting, explanation and progress through movement of the door character and its handle. This animation has not yet been implemented or tested.